Identifying LLM Families from Their Responses

Team Leader: Jethin Grandhi

Anushka Shrivastava

Shon Zou

CMPSC 448 Midterm Project Report

Instructor: Prof. Wenpeng Yin

October 11, 2026

Abstract

We tested whether a small neural network (both convolutional and recurrent neural networks) could tell if a response was written by Claude, GPT, or Gemini using 8,121 responses from LMArena. Both the RNN and CNN models achieved around 90% accuracy, compared to 33.3% chance baseline. User prompts alone provided almost no predictive power, and adding prompts to responses did not meaningfully improve classification accuracy. The models also showed strong performance across our domains, coding and non-coding conversations. Interestingly, formatting, such as punctuation and markdown, turned out to be the biggest indicator for a classifier: even with all words hidden, the CNN achieved 89.7% accuracy, dropping to 75.5% when formatting was removed.

Abstract
I – Problem Definition
II – Dataset
    2.1 Source
    2.2 Construction
III – Models and Training
    3.1 Token Processing
    3.2 Models
    3.3 Training
IV – Results & Analysis
    4.1 RQ1: Can we identify the family from a response?
    4.2 RQ2: Does the prompt help in classification?
    4.3 RQ3: Do the fingerprints generalize across domains?
        General fingerprint or task-specific patterns?
        Are some families more consistently identifiable?
    4.4 RQ4: What distinguishes the families?
V – Experiment Limitations
VI – Lessons Learned
VII - Citations
VIII – AI Integrity Statement

I – Problem Definition

Our goal is to identify which LLM family – Claude (Anthropic), GPT (OpenAI), or Gemini (Google) – generated a given response. We treat this as a three-class classification problem. Since the filtered dataset we tested on is balanced across the three families, random guessing gives 33.3% accuracy. We use accuracy as our main metric, along with precision, recall, F1 scores for each family, and confusion matrices.

We investigate all four research questions of the project (Table 1):

Table 1	**Question**	**What we did**
RQ1	Can we identify which LLM family generated a response?	Trained a CNN and an RNN on the response text.
RQ2	Does the user's prompt help?	Trained the same models on the prompt only, the response only, and both.
RQ3	Do the fingerprints generalize across domains?	Trained on non-coding conversations and tested on coding ones, and the reverse.
RQ4	What distinguishes the families?	Measured style statistics per family, and retrained the CNN with one kind of signal removed from the responses.

II – Dataset

2.1 Source

We use the public **lmarena-ai/arena-human-preference-140k** dataset from Hugging Face. Each record represents a vote on the LMArena platform, where a user submits a prompt, two anonymous models generate responses side by side, and the user votes for the better answer. Each record includes the names of both models, their full conversations, and additional details such as language, number of turns, and an **"is_code**" flag indicating whether the conversation involves coding.

This dataset is a good fit for our project because each record gives us two sets of the information we need: the LLM name, its input, and its output. Since both models answer the same prompt, we can also use this setup for RQ2.

According to the dataset card, user prompts are licensed under CC-BY-4.0, while model-generated responses follow the terms of use of their respective providers. We use the data only for this course project. The original dataset is not stored in our repository; instead, **src/preprocessing.py** downloads the source file and rebuilds our processed dataset.

The full dataset is divided into seven files. We use only the first file, which contains 19,377 of the approximately 135,600 votes, as it already provides an ample amount of responses for our experiments.

2.2 Construction

Table 2 lists every step from the source file to our dataset.

Table 2 — Data Preprocessing & Sample Counts	
Step	**Records Left**
Votes in the source file	19,377
Keep English conversations	10,226
Keep single-turn conversations	8,759
Split each vote into two rows; drop rows with an empty prompt or response	17,513
Remove exact duplicates	17,511
Keep the three largest families	9,170
Downsample every family to the size of the smallest	8,121

Why these filters: English only, so that one tokenizer works for all text and the classifier cannot use the language as a shortcut. Single-turn only, so that every row is exactly one input and one output. *It should be noted that some entries in the dataset are incorrectly marked as English*, though we determined given how rare they were, that the effects on the final results were negligible.

Families: We map each of the model names in the data to a family by its name (for example claude-opus-4-20250514 to Claude, o3-2025-04-16 and gpt-4.1-2025-04-14 to GPT). This gave 14 families, the top three being Claude: 3,632 responses, GPT: 2,831, and Gemini: 2,707. Every model name in the dataset matched a family. We kept the three largest families, because balancing them discards the least data.

Balancing: We randomly downsampled Claude and GPT to Gemini's 2,707 responses

with a fixed seed.

Table 3 shows the result. Note that a label is a family, not a single model.

Table 3 — Distribution of Models To Be Processed			
Family	**Responses**	**Non-Coding**	**Coding**
Claude	2,707	1,872	835
GPT	2,707	1,783	924
Gemini	2,707	1,793	914
Total	8,121	5,448	2,673

Split: We split by *prompt*, not by row: the 8,121 responses come from 6,171 distinct prompts, and every response follows its prompt into 80% training, 10% validation or 10% test. This gives 6,507 training, 805 validation and 809 test responses, and no test prompt is ever seen in training.

III – Models and Training

3.1 Token Processing

We lowercase the text and split it into words, single punctuation/symbol characters, and line breaks, so that the models can see markdown and layout ("## Summary" becomes #, #, summary). We keep the first 256 tokens. The median response has 526 tokens, so for ~71% of responses the models see only the beginning. The vocabulary used is the 20,000 most frequent training tokens. For RQ2 we also build a prompt-only input, and a prompt + response input.

3.2 Models

Table 4 below provides both architectures. The CNN detects patterns of six consecutive tokens, such as a line break followed by ##, and uses max pooling to capture whether each

pattern appears anywhere in the text. The RNN processes the text from both directions and takes the maximum value of each hidden unit across all positions, allowing it to capture important information from anywhere in the response.

Table 4 – Model Architecture / Specifications		
Model	**CNN**	**RNN**
Embedding	20,000 tokens, 128 dimensions	20,000 tokens, 128 dimensions
Feature Extraction	1-D convolution, 128 filters of width 6, ReLU activation function	One bidirectional LSTM layer, 128 hidden units per direction
Pooling	Maximum over all positions (128 values)	Maximum over all positions (256 values, forward & backward)
Output	Dropout 0.5, linear layer 128 → 3 output scores	Dropout 0.5, linear layer 256 → 3 output scores

3.3 Training

All runs use cross-entropy loss, Adam (learning rate 0.001), batch size 64, an epoch count of 10, and seed 0 (split, initialization and batch order) – on a CPU with PyTorch 2.9. We keep the weights of the epoch with the best validation accuracy, which levels off near 92% after about six epochs and then moves by a point or two. We did not run a systematic hyperparameter search.

All thirteen training runs take about 15 minutes; `python src/preprocessing.py` then `python src/experiments.py` reproduces every number here.

IV – Results & Analysis

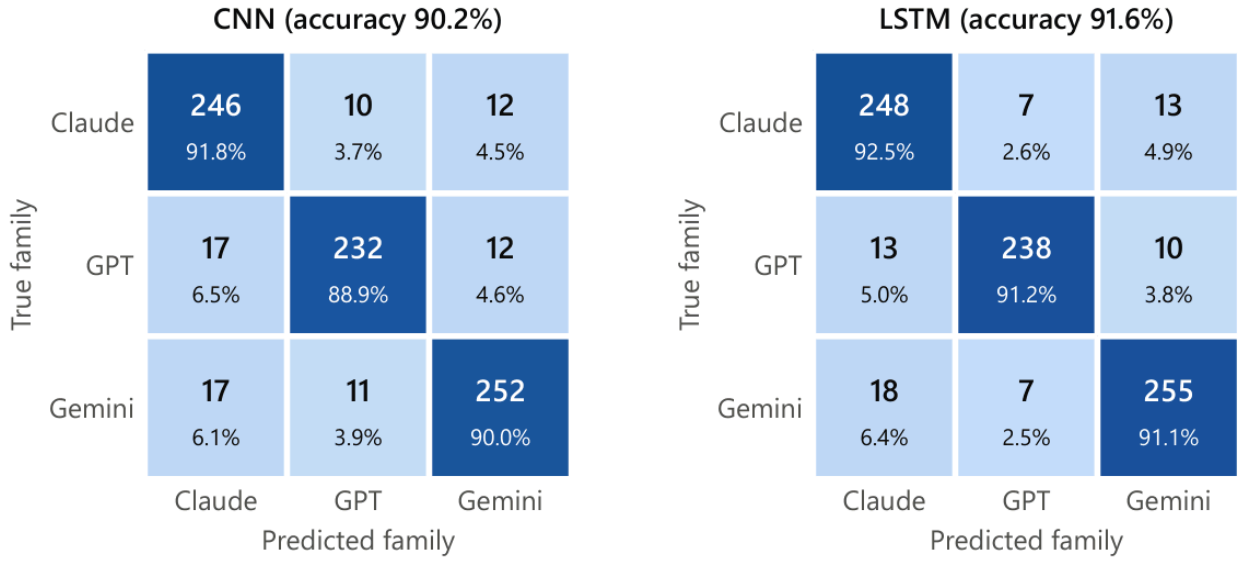
4.1 RQ1: Can we identify the family from a response?

The RNN achieved 91.6% accuracy compared with 90.2% for the CNN, a difference of 1.4 percentage points (about 11 additional correct predictions out of 809). Because we did not conduct a statistical significance test, we do not conclude that either architecture is superior. No

family is much harder than the others: F1 lies between 0.90 and 0.93 for every family in both models. The errors lean slightly towards Claude: for the CNN, 34 GPT or Gemini responses were predicted as Claude, while only 22 Claude responses were predicted as another family, which gives Claude the highest recall and the lowest precision in both models.

Table 5	CNN (accuracy 90.2%)			RNN/LSTM (accuracy 91.6%)		
Family	**Precision**	**Recall**	**F1**	**Precision**	**Recall**	**F1**
Claude	0.879	0.918	0.898	0.889	0.925	0.907
GPT	0.917	0.889	0.903	0.944	0.912	0.928
Gemini	0.913	0.900	0.906	0.917	0.911	0.914

The figure below shows the confusion matrices for both models, illustrating how often each family is correctly identified and which families are most commonly confused with one another.



4.2 RQ2: Does the prompt help in classification?

In short, no. Given only the prompt, models score almost entirely by chance, with loss during training consistently staying above 0.7 and the validation accuracy staying within the 30 – 36% range. This is expected: on LMArena the platform, not the user, assigns two models to each prompt and both answer the same text, so the prompt says nothing about who is answering. Beating the 33% chance would have pointed to an artifact, such as a prompt appearing in both training and test data, which our prompt-level split rules out.

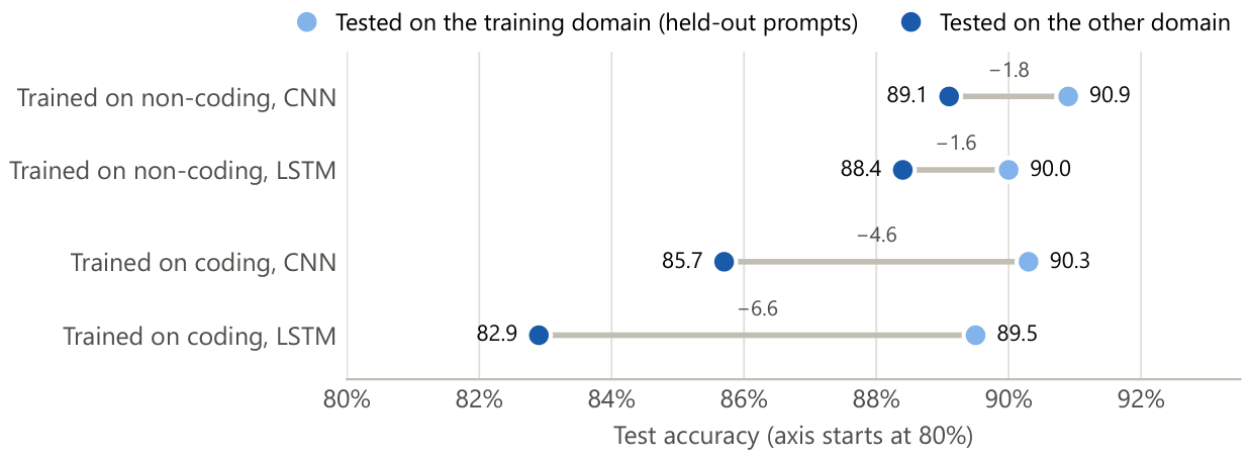
Adding a prompt to a response changes nothing we can measure: the changes from the response alone (+0.4 and −2.2 points) are within the noise of 809 test responses and point in opposite directions.

4.3 RQ3: Do the fingerprints generalize across domains?

Using the "is_code" flag as our domain, for each direction we split the training domain by prompt as before (80/10/10), train on its training part, and test on that domain's held-out test part and on *all* responses of the other domain.

The performance does not decrease much on unseen tasks, especially for models trained on non-coding conversations, where both models lose under two points on coding conversations. However, models trained on coding conversations seem to fare worse, losing 4.6 (CNN) and 6.6 (RNN) points on non-coding conversations. Their accuracy is still above 82%, however. As the training set for coding is smaller, the experiment can't determine whether this difference is from training-set size or domain-specific patterns.

The figure below compares their accuracies tested on their training domain compared to the other domain.



General fingerprint or task-specific patterns?

Mostly a general fingerprint, with a smaller task-specific part. The coding-trained models may transfer worse because they had half as much training data (2,133 against 4,336 responses), or because coding conversations show a narrower range of each family's style; this experiment cannot separate the two.

Are some families more consistently identifiable?

The differences are small (Table 6), but GPT is the most stable: its F1 is the highest, or within 0.002 of the highest, in all four tests, and its precision stays between 0.92 and 0.96. Most of what is lost across domains is responses wrongly labelled as Claude or Gemini. In the largest drop (RNN, coding to non-coding), 58% of all errors are GPT or Gemini responses being labelled as Claude. A plausible reading is that a response showing none of the formatting markers the classifier learned tends to be assigned to Claude, the plainest LLM family.

Table 6 – F1 Scores	CNN			RNN/LSTM		
Train → Test	**Claude**	**GPT**	**Gemini**	**Claude**	**GPT**	**Gemini**
Non-Coding → Coding	0.866	0.903	0.905	0.867	0.900	0.884
Coding → Non-Coding	0.849	0.868	0.854	0.823	0.842	0.825

4.4 RQ4: What distinguishes the families?

We first measured simple properties of all 8,121 responses (Table 7), then retrained the CNN on responses with one kind of signal removed (Table 8).

Length differs a lot: Gemini's median response is 3.5 times as long as Claude's, and GPT's about twice as long. Vocabulary richness does not differ, whereas formatting differs sharply. GPT has typographic habits the others rarely show: curly quotes in 72% of its responses (almost never for Claude), em dashes in 46% and emoji in 35%. Gemini uses bold text in 89% of its responses and bullet lists in 77%. Claude is the plainest: it has the lowest share on seven of the eight features and ties Gemini on the eighth (emojis).

Given that the statistics could only show what differs and not what a classifier actually uses, we used **ablations** (removing features of the input text) in three different ways, shown in Table 8 below, using it to retrain the CNN specifically from scratch each time. The results indicate the following:

- **The classifier does not rely on responses naming themselves.** About 9% of responses mention a model or company name; masking them changes nothing.
- **Formatting carries most of the signal.** With every word hidden, the CNN is within half a point of the full model; removing the formatting costs almost 15 points.
- **Words carry some signal.** Words alone still reach 75.5%, so the families also differ in wording, but adding the words to the formatting gains only half a point.

While the signals are both linguistic and structural, the structural ones dominate. Stripping markdown and normalizing quotes and dashes would erase much of the fingerprint; the words-only result suggests about 75% would remain. Overall, the results indicate that structural

features, such as Markdown, punctuation, and quotation styles, are strong indicators of model family, although word choice also contributes useful information.

Table 7 – Style Statistics Per Family			
Measure	**Claude**	**GPT**	**Gemini**
Linguistic Signals			
Median words per response	179	345	626
Median words per sentence	23.8	17.7	16.7
% Unique words among the first 100 words	73.2%	74.1%	72.2%
Structural Signals			
% Bold text	38.6%	70.9%	88.6%
% Headers	19.9%	50.7%	35.0%
% Bulleted lists	47.1%	70.9%	77.0%
% Tables	1.8%	22.2%	7.9%
% Horizontal rules	2.9%	49.6%	28.3%
% Em dashes (— vs -)	3.2%	45.5%	5.7%
% Curly quotes (“ ” v.s. ")	0.0%	71.9%	9.0%
% Emojis	3.4%	34.6%	3.3%

Table 8 – CNN Test Accuracy After Feature Ablation		
Responses given to the CNN	**Test Accuracy**	**Change**
Original (RQ1 result)	90.2%	
Model & company names replaced with the word “model”	90.7%	+ 0.5%
Words only: punctuation, symbols, and line breaks removed	75.5%	– 14.7%
Formatting only: each word replaced by the same placeholder	89.7%	– 0.5%

V – Experiment Limitations

- **Data coverage:** We only used the first source file of the dataset to cut down on processing time, so results may not generalize to every conversation in the dataset.
- **Families, not models:** We did not test whether models in each family can be told apart.
- **A narrow slice of data:** We selected only English and single-turn conversations, and so classification scores or found formatting habits may not apply elsewhere,
- **Truncation:** Many responses are cut at 256 tokens, and so the models do not analyze most of the full text in longer responses.
- **Uneven domain comparison:** in RQ3, the coding-trained models had half the training data, and the own-domain test sets (277 and 550) are relatively small.

VI – Lessons Learned

- **Preprocessing mattered more than the architecture**. Our first tokenizer kept only words and scored about 15 points lower, and keeping formatting was our largest improvement. In contrast, CNN versus RNN never moved the accuracy by more than 2.8 points.
- **Accuracy does not explain itself**. While there was certainly a difference between models, it took the usage of ablations and style statistics to figure out what exactly the classifier was seeing that was differentiating between the different models.
- **The smallest of bugs can cause the biggest time loss**. By default pandas read “NA” as a missing value (not a string), which crashed the tokenizer; and denormal floats (extremely small numbers) made RNN epochs on padded prompts five times slower until a PyTorch setting (`torch.set_flush_denormal(True)`) fixed it.

- **Communication is key for repository management**. Such mishaps include accidentally publishing the entire generated 400,000-line data.csv file into the repository, which had to be cleaned up via git commands. Two people had also worked on preprocessing.py at the same time, resulting in an hour spent properly merging the two versions of the file.

VII - Citations

1. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.

2. LMArena. (2025). arena-human-preference-140k [Data set]. Hugging Face.

3. McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 56–61.

4. Wenpeng Yin, Chapter 6: LLM Pretraining

5. Wenpeng Yin, Chapter 7: LLM Instruction Tuning

6. Wenpeng Yin, Chapter 8: Tuning LLMs with Human Preference Data

7. Wenpeng Yin, Chapter 9: ML Training--Gradient descent

VIII – AI Integrity Statement

We used **Claude** as a supplementary tool throughout this project for the following purposes:

- Assisting in narrowing down a suitable dataset based on the project requirements.
- Clarifying the practical implementation and functionality of LSTM and CNN architectures.
- Identifying and resolving bugs, including a preprocessing script crash encountered when running the project from a fresh clone.
- Assisting in the interpretation of experimental results.
- Generating figures for the report using the experimental results.

All major project decisions, including dataset selection, the design of the preprocessing and processing pipelines, model architectures, and experimental methodology, were independently made by our team. Claude was used solely as a supporting resource and did not replace our own analysis or decision-making.

No other AI tools were used during this project.